

Evaluating TurboQuant-Family Compression on CLIP Multimodal Embeddings

CS 639: Introduction to Foundation Models — Final Project Report

A. Agarwal (aagarwal235) A. Kumar (akumar323) C. Jain (cajain)
R. Singh (rsingh) S. Agarwal (sagarwal) S. Barjatya (sbarjatya)
S. Datta (sdatta9)

Spring 2026

Abstract

CLIP-style multimodal retrieval systems store large databases of image and text embeddings, making vector compression a practical necessity. Recent analytic quantizers such as QJL, PolarQuant, and TurboQuant were developed primarily for Transformer KV-cache compression and high-dimensional vector quantization, but their behavior on multimodal embeddings is less understood. We evaluate QJL, PolarQuant, TurboQuant, and FAISS product quantization on 512-dimensional CLIP ViT-B/32 embeddings from Flickr30k across four retrieval tasks: text-to-image, image-to-text, text-to-text, and image-to-image. Our main finding is that cross-modal retrieval is much more fragile under compression than same-modal retrieval. At 1 bit per dimension, same-modal image retrieval remains relatively strong, while text-to-image recall collapses. Among compressors, FAISS-PQ and PolarQuant form the strongest memory-accuracy frontier in the 2–4 bit regime. TurboQuant improves substantially with bit-width and is much stronger than plain QJL at high compression, but it does not exceed PolarQuant or FAISS-PQ on CLIP retrieval. These results suggest that near-optimal distortion guarantees for generic vectors or KV caches do not automatically translate into optimal cross-modal retrieval performance.

1 Introduction

CLIP embeds images and text into a shared vector space where retrieval is performed by inner product or cosine similarity. This representation is now widely used in multimodal search systems, but it creates an immediate storage problem: a database with millions or billions of 512-dimensional float32 vectors becomes expensive to store, transfer, and search. Compressing database embeddings while leaving queries in full precision is therefore an attractive post-training optimization.

Most standard vector compression methods, such as product quantization (PQ), are learned from the database distribution. In contrast, recent quantizers developed for long-context language model inference aim to be online, lightweight, and theoretically grounded. QJL uses a random Johnson–Lindenstrauss projection followed by sign-bit quantization and an asymmetric inner-product estimator. PolarQuant uses random preconditioning and recursive polar coordinates to quantize angles efficiently. TurboQuant combines an MSE-optimized rotated-coordinate quantizer with a QJL residual correction to obtain an unbiased inner-product estimator.

These methods are appealing for CLIP databases because they are fast to apply and do not require heavy index training. However, CLIP embeddings are not ordinary unimodal vectors. Image and text embeddings occupy systematically shifted regions of the shared space, often described as a modality gap. Cross-modal retrieval depends on this geometric alignment being preserved after compression. A compressor that preserves average Euclidean distortion may still damage the ordering of the nearest image-text pairs.

Hypotheses. Our project began from the following two hypotheses, motivated by the proposal. First, we predicted that TurboQuant-family methods, QJL, and PolarQuant would compress CLIP embeddings to roughly 3–4 bits per dimension with acceptable retrieval quality, because random rotation and sketching should preserve much of the high-dimensional geometry. Second, we predicted that cross-modal tasks, text-to-image and image-to-text, would degrade more sharply at low bit-widths than same-modal tasks, text-to-text and image-to-image. The reason is that cross-modal retrieval depends on preserving the alignment between image and text embedding distributions, while same-modal retrieval mostly depends on preserving local neighborhoods within one distribution.

We therefore test two concrete claims:

H1: Cross-modal retrieval degrades faster under compression than same-modal retrieval.

H2: TurboQuant-family analytic compressors can be competitive with trained PQ on CLIP retrieval at 2–4 bits per dimension.

Our results strongly support H1. H2 is only partially supported: analytic methods are competitive, especially PolarQuant, but FAISS-PQ remains the strongest low-bit baseline and TurboQuant does not dominate PolarQuant on CLIP.

2 Related Work

CLIP and multimodal retrieval. CLIP learns a joint image-text embedding space using contrastive training, enabling retrieval by comparing normalized image and text embeddings [1]. The same property that makes CLIP useful also makes it sensitive to geometry: retrieval depends not only on preserving within-modality neighborhoods, but also on preserving image-text alignment.

Product quantization. Product quantization partitions a vector into subvectors and learns a codebook for each subspace [2, 3]. It is widely used in approximate nearest-neighbor search because it gives a strong accuracy-memory tradeoff. Its drawback for online settings is that it requires codebook training, and its performance depends on the database distribution.

QJL. QJL applies a Gaussian random projection to database vectors and stores only the sign of each projected coordinate [4]. For a query q and database vector x , the estimator

$$\widehat{\langle q, x \rangle} = \frac{\sqrt{\pi/2}}{m} \|x\|_2 \langle Sq, \text{sign}(Sx) \rangle$$

is unbiased when S has i.i.d. Gaussian entries. This is attractive for asymmetric retrieval because the query remains full precision while database vectors are compressed.

PolarQuant. PolarQuant first applies random preconditioning, then recursively converts Cartesian coordinates into polar coordinates and quantizes the resulting angles [5]. After preconditioning, the paper derives predictable angle distributions, which motivates scalar angle quantization without per-block normalization constants. In our implementation, we preserve the recursive polar transform and random rotation, but fit per-level Lloyd-Max codebooks empirically on the rotated CLIP embeddings.

TurboQuant. TurboQuant targets near-optimal vector quantization rates [6]. Its MSE stage applies a random rotation and scalar-quantizes rotated coordinates using Lloyd-Max centroids for the coordinate distribution of a point on the unit sphere. Its inner-product version then applies QJL to the residual. We implement this two-stage structure for CLIP retrieval: total bit-width b uses a $(b - 1)$ -bit TurboQuantMSE stage and a 1-bit QJL residual.

Community implementations and recent evidence. Because TurboQuant is recent, community implementations are evolving quickly. The `tonbistudio/turboquant-pytorch` project reports practical variants of TurboQuant-style quantization in PyTorch and discusses settings where QJL residuals may be delicate in generation workloads. Memoriant’s DGX Spark KV-cache benchmark and `llama.cpp` discussions emphasize a related point: memory savings alone are not enough, because dequantization and attention-time overheads can dominate at long context lengths. These observations are complementary to our work. They study LLM KV-cache inference, while we study CLIP retrieval. However, they reinforce the same broader lesson: a theoretically appealing quantizer must be evaluated in the downstream computation where it is used.

3 Methodology

3.1 Dataset, embeddings, and hardware

We use the Hugging Face parquet mirror of Flickr30k, containing 31,014 images and 155,070 captions, with five captions per image. Images and captions are embedded with frozen CLIP ViT-B/32 into 512-dimensional vectors. All vectors are L2-normalized, so inner product equals cosine similarity. The embedding step was run once on Google Colab with GPU acceleration. The downstream compression, retrieval, profiling, plotting, and analysis steps were run locally on a MacBook Pro with an M1 Pro chip and 16GB RAM.

This split mirrors the intended deployment setting: model inference to produce embeddings is expensive and GPU-friendly, while indexing and retrieval experiments can be run independently once embeddings are saved.

3.2 Retrieval tasks

We evaluate four tasks using Recall@ k for $k \in \{1, 5, 10\}$:

- **T1 text→image:** a caption queries the image database. The correct target is its paired image.
- **T2 image→text:** an image queries the caption database. Any of its five captions counts as correct.
- **T3 text→text:** a caption queries the caption database. The correct set is the other four captions of the same image; self-retrieval is excluded.
- **T4 image→image:** an image queries the image database. Since Flickr30k has no duplicate image labels, we evaluate nearest-neighbor preservation by using the float32 top-1 image neighbor after excluding the query image itself.

T1 and T2 are cross-modal tasks; T3 and T4 are same-modal controls. We report Recall@1, Recall@5, and Recall@10 because they capture different retrieval regimes. Recall@1 measures whether the top result is exactly correct, Recall@5 reflects a small user-facing retrieval set, and Recall@10 is a more stable measure of whether the correct item remains near the top under compression. The plots and main table emphasize Recall@10 because it is less noisy than Recall@1 while still sensitive to ranking degradation.

3.3 Compressors and implementation details

Table 1 summarizes the evaluated methods. Queries always remain full precision; only database vectors are compressed. This is an asymmetric retrieval setting: the database is stored compactly, while each query is processed at full precision.

Table 1: Compressors evaluated.

Method	Description	Bits tested
Uncompressed	float32 reference	32
QJL	Gaussian sign sketch with asymmetric estimator	1, 2, 3, 4, 8
PolarQuant	recursive polar transform, per-level Lloyd-Max angle codebooks	1, 2, 3, 4
TurboQuant	TurboQuantMSE($b - 1$) + 1-bit QJL residual	1, 2, 3, 4
FAISS-PQ	FAISS IndexPQ trained codebooks	1, 2, 4, 8

QJL. For QJL, we sample a Gaussian matrix $S \in \mathbb{R}^{m \times d}$ and store $\text{sign}(Sx)$ for each database vector. The parameter m determines the bit budget: $m = d$ gives 1 bit per dimension, $m = 2d$ gives 2 bits per dimension, and so on. At query time, we compute Sq and use the paper’s asymmetric estimator. Because our CLIP vectors are unit-normalized, standalone QJL could omit storing norms; however, our implementation stores one float32 norm per vector to support the same code path for non-unit TurboQuant residuals. This makes memory accounting slightly conservative for standalone QJL.

PolarQuant. For PolarQuant, we apply a shared random orthogonal rotation, recursively convert pairs of coordinates into polar radii and angles, quantize the angles, and reconstruct approximate database vectors for scoring. The paper derives analytic angle distributions after random preconditioning. Our implementation follows the random-preconditioned polar transform but fits per-level Lloyd-Max codebooks empirically on the rotated CLIP angles. This adapts the method to CLIP while preserving the main polar-coordinate structure of the paper.

TurboQuant. For TurboQuant, total bit-width b is implemented as a $(b - 1)$ -bit TurboQuantMSE stage plus a 1-bit QJL residual. TurboQuantMSE rotates each vector, scalar-quantizes rotated coordinates using Lloyd-Max centroids for the sphere-coordinate distribution, and rotates the reconstruction back. Then QJL is applied to the residual $r = x - \hat{x}_{\text{MSE}}$. At query time, the final score is

$$\langle q, \hat{x}_{\text{MSE}} \rangle + \widehat{\langle q, r \rangle}_{\text{QJL}}.$$

This follows the paper’s logic: the MSE stage keeps the residual small, and QJL makes the residual inner-product correction unbiased.

FAISS-PQ. For FAISS-PQ, we use 8-bit codebooks and choose the number of subquantizers M so that $M \cdot 8/512$ equals the target bits per dimension. Thus 1, 2, 4, and 8 bits per dimension correspond to $M = 64, 128, 256, 512$, respectively. These bit-widths were chosen to match the common powers-of-two compression grid used by the analytic methods and to make memory-accuracy comparisons simple. The consequence is that the subvector dimension decreases from 8 at 1 bit/dim to 1 at 8 bits/dim. This is not the only possible PQ configuration, but it gives a controlled trained baseline at the same nominal bit budgets.

3.4 Bit-widths, seeds, and metrics

We evaluate bit budgets $\{1, 2, 3, 4, 8, 32\}$ where applicable. The 1–4 bit regime is the main focus because it is where aggressive compression is useful and where the TurboQuant-family papers report strong low-bit behavior. 8-bit QJL and FAISS-PQ are included as high-quality compressed references, and 32-bit float storage is the uncompressed upper baseline. PolarQuant and TurboQuant are reported up to 4 bits because this already approaches the uncompressed ceiling for several tasks, and higher-bit empirical codebook quantization becomes less informative for our main cross-modal question.

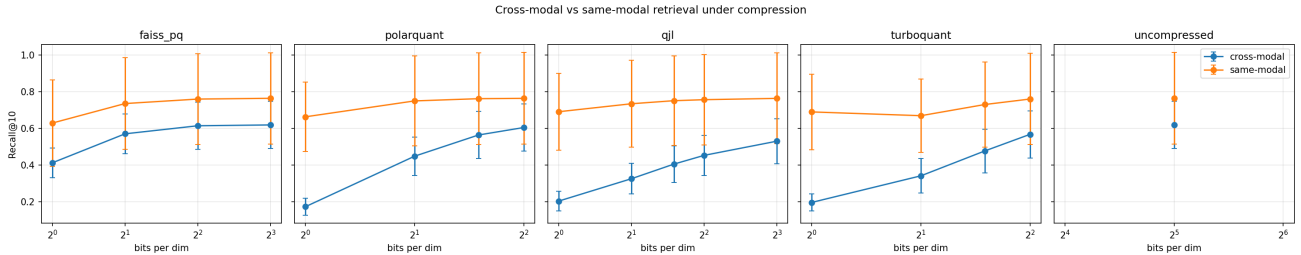


Figure 1: Recall@10 under compression, averaged into cross-modal and same-modal groups. Error bars are standard deviations across five seeds. Cross-modal retrieval consistently degrades faster than same-modal retrieval.

Each method-bit setting is run over five random seeds. Seeds control random projections, rotations, and codebook initialization. We use multiple seeds because QJL, PolarQuant, and TurboQuant are randomized algorithms; a single seed could overstate or understate performance. We report means across seeds and use seed standard deviation as error bars.

3.5 Implementation validation

Before running CLIP experiments, we validated the compressor implementations on synthetic 512-dimensional unit vectors. This matters because downstream retrieval results are only meaningful if the compressors first satisfy basic mathematical expectations.

The most important validation is the TurboQuantMSE stage. TurboQuantMSE is the first stage of TurboQuant: it rotates the input and scalar-quantizes rotated coordinates. The TurboQuant paper reports small-bit MSE distortion values for unit vectors, so reproducing those values is a direct sanity check that the MSE stage is implemented correctly. For bit-widths 1 through 4, we obtain:

$$0.3627, \quad 0.1171, \quad 0.0344, \quad 0.0095.$$

These match the expected distortion scale. We also check that QJL, PolarQuant, and TurboQuant produce finite inner-product estimates and that correlation with exact inner products improves as bit-width increases. These tests do not prove downstream retrieval quality, but they rule out estimator-scale or rotation-codebook bugs before the full sweep.

4 Results

4.1 Main retrieval results

Table 2 reports Recall@10 averaged over five seeds. The strongest result is the gap between cross-modal and same-modal robustness. At 1 bit/dim, all methods retain much more same-modal performance than cross-modal performance. For example, QJL at 1 bit obtains 0.889 on T4 image-neighbor preservation but only 0.154 on T1 text-to-image retrieval.

H1: cross-modal fragility. H1 is strongly confirmed. Cross-modal tasks T1 and T2 degrade faster under compression than same-modal tasks T3 and T4. This is visible in Figure 1: for every compressor, the blue cross-modal curve lies below the orange same-modal curve, especially at low bit-widths. This suggests that preserving same-modal nearest-neighbor structure is not sufficient for evaluating CLIP compression.

Table 2: Recall@10 averaged over five seeds and 1,000 queries per task. Best compressed method in each bit group and task is bolded.

Method	Bits	T1 txt→img	T2 img→txt	T3 txt→txt	T4 img→img
Uncompressed	32	0.498	0.741	0.528	1.000
FAISS-PQ	1	0.337	0.488	0.407	0.851
QJL	1	0.154	0.255	0.493	0.889
PolarQuant	1	0.132	0.215	0.484	0.842
TurboQuant	1	0.156	0.238	0.495	0.884
FAISS-PQ	2	0.468	0.673	0.499	0.971
QJL	2	0.248	0.404	0.510	0.958
PolarQuant	2	0.350	0.547	0.518	0.982
TurboQuant	2	0.254	0.430	0.480	0.858
FAISS-PQ	4	0.493	0.736	0.525	0.994
QJL	4	0.350	0.556	0.524	0.989
PolarQuant	4	0.483	0.726	0.527	1.000
TurboQuant	4	0.447	0.689	0.525	0.995

H2: TurboQuant-family transfer. H2 is partially supported but not fully confirmed. TurboQuant improves with bit-width and is much stronger at 4 bits than at 1–2 bits. However, it does not beat PolarQuant or FAISS-PQ on CLIP retrieval. At 4 bits, TurboQuant reaches 0.447 on T1, while PolarQuant reaches 0.483 and FAISS-PQ reaches 0.493. This suggests that TurboQuant’s strong distortion behavior does not automatically make it the best compressor for CLIP’s cross-modal neighborhood structure.

4.2 Memory-accuracy tradeoff

Figure 2 collapses the four retrieval tasks into mean Recall@10 and plots it against ideal bytes per vector. FAISS-PQ is strongest at the very low-byte end and remains excellent at 2–4 bits. PolarQuant joins the frontier around 2 bits and nearly matches FAISS-PQ at 4 bits. QJL is fast and simple but is dominated in accuracy. TurboQuant occupies the middle: better than QJL at high compression, but not better than PolarQuant.

Mean Recall@10 at 4 bits illustrates the ranking:

$$\text{FAISS-PQ} = 0.687, \quad \text{PolarQuant} = 0.684, \quad \text{TurboQuant} = 0.664, \quad \text{QJL} = 0.605.$$

Thus, if the goal is highest CLIP retrieval quality at 2–4 bits, the best options are still FAISS-PQ and PolarQuant.

4.3 Embedding geometry

Figure 3 helps explain why the analytic quantizers work at all, but also why cross-modal retrieval remains fragile. Before rotation, CLIP coordinates are highly non-Gaussian: image coordinates have kurtosis about 69.4 and text coordinates about 36.7. After random orthogonal rotation, both skew and kurtosis are near zero. This supports the theoretical motivation behind random-rotation methods such as PolarQuant and TurboQuant: rotated coordinates behave much more like Gaussian or sphere-coordinate samples.

However, rotation does not eliminate the modality gap. The cosine between the mean image embedding and mean text embedding is only 0.344, and the norm of the gap vector is about 0.802. Because orthogonal rotations preserve distances and angles, this gap remains after rotation. Compression noise

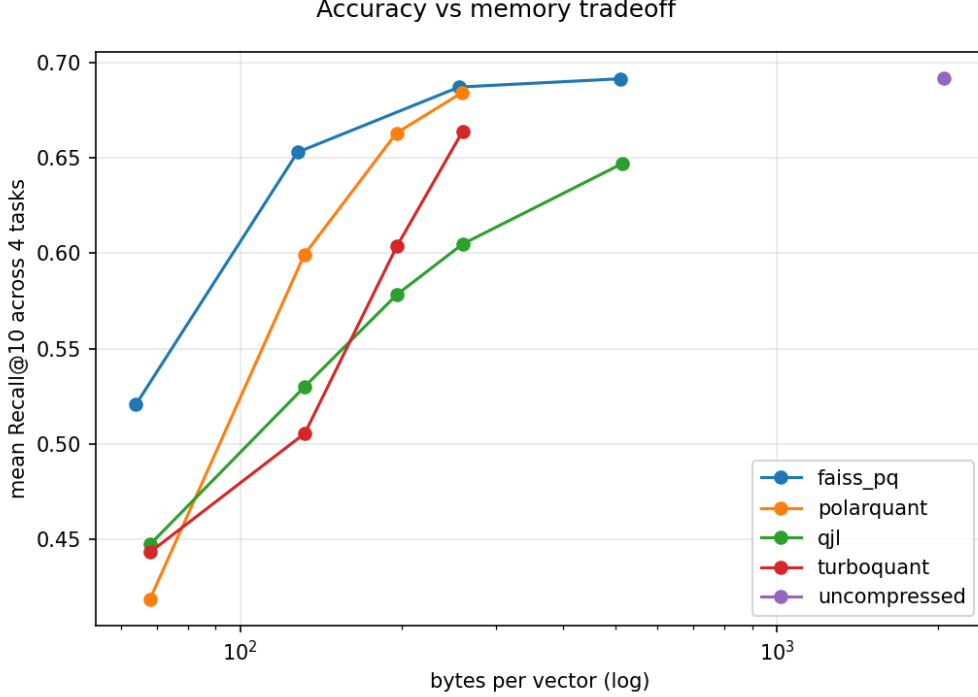


Figure 2: Mean Recall@10 across all four tasks versus ideal bytes per vector. FAISS-PQ and PolarQuant form the strongest memory-accuracy frontier; TurboQuant is competitive but does not dominate.

can therefore preserve local same-modal neighborhoods while still damaging the finer cross-modal alignment needed for text-image matching.

5 Discussion

Why does FAISS-PQ remain strong? FAISS-PQ learns codebooks from the actual CLIP embedding distribution. This gives it an advantage over data-oblivious analytic methods, especially at 1–2 bits where every bit must capture dataset-specific structure. The downside is training cost: PQ requires a codebook training phase, while QJL and TurboQuant-style random projections can be applied almost immediately once the random matrix or rotation is fixed.

Why does PolarQuant work well? PolarQuant benefits from random rotation, which Gaussianizes CLIP coordinate marginals, and from polar angle quantization, which appears well matched to normalized embeddings. Its empirical per-level Lloyd-Max codebooks adapt to CLIP while preserving the main random-preconditioned polar structure of the paper. This likely explains why PolarQuant nearly matches FAISS-PQ at 4 bits.

Why does TurboQuant not dominate? TurboQuant is designed to reduce MSE first and then correct residual inner products with QJL. This gives strong synthetic distortion behavior, as our validation tests confirm. But CLIP retrieval is not only a generic MSE problem. Cross-modal retrieval depends on preserving the relative ordering of image-text pairs near the decision boundary. The QJL residual correction adds unbiasedness but also variance, and this variance may not improve top- k ranking when the base quantizer already preserves most useful structure.

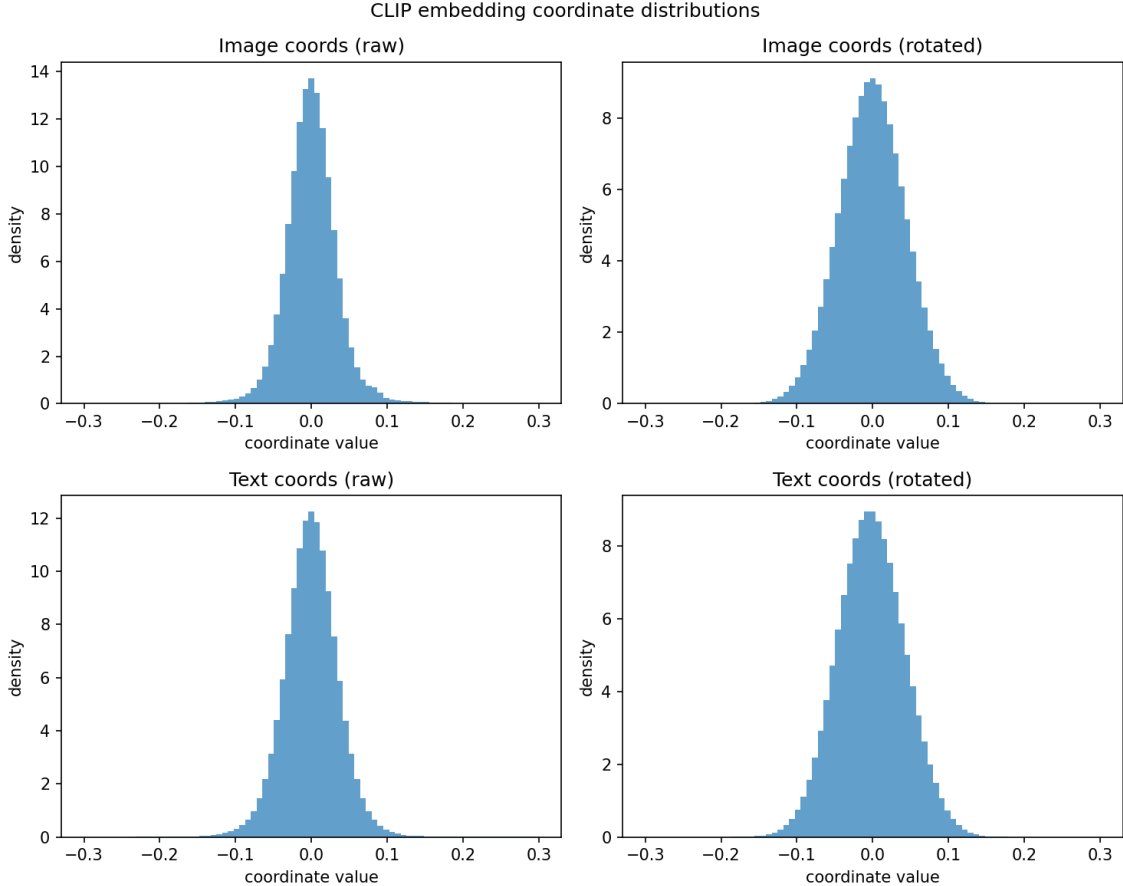


Figure 3: Coordinate distributions before and after random rotation. Rotation makes image and text coordinate marginals approximately Gaussian, but it does not remove the modality gap.

What should practitioners take away? For CLIP databases, the right metric is not just reconstruction MSE or same-modal nearest-neighbor preservation. A compressor should be evaluated directly on text-to-image and image-to-text recall. In our experiments, same-modal tasks remain relatively robust under aggressive compression, but cross-modal tasks degrade sharply. A deployment that only tests image-to-image retrieval could therefore overestimate compression quality.

Relation to community findings. Recent community implementations of TurboQuant for KV-cache compression report that QJL residuals can be delicate in practice, particularly when downstream attention applies a softmax. Hardware-focused benchmarks also show that KV-cache quantization must balance memory savings against dequantization overhead. Our setting is different: CLIP retrieval uses dot-product ranking rather than autoregressive attention. Still, our results agree with the broader theme that compression quality must be measured in the final downstream operation, not only by theoretical distortion.

6 Limitations

- **Implementation fidelity.** Our TurboQuant MSE stage matches the paper’s reported small-bit distortion scale, but our PolarQuant uses empirical per-level Lloyd-Max codebooks rather than purely analytic precomputed angle codebooks. This makes our PolarQuant partly data-adaptive. That may

improve CLIP performance, but it weakens the claim that it is fully online and data-oblivious in the same sense as the paper.

- **Memory accounting.** We report ideal bytes per vector from each compressor’s bit accounting. This is the right comparison for algorithmic compression, but it is not the same as measuring a production index. Python dictionaries, unpacked NumPy arrays, and temporary decoded matrices can use much more memory than an optimized packed implementation. Conversely, standalone QJL on unit-normalized CLIP embeddings could omit the stored norm and save 4 bytes per vector. Therefore, our memory-accuracy plot should be read as an idealized storage comparison, not a systems benchmark.
- **Runtime interpretation.** Our code is a reference implementation intended for correctness and comparison, not a fused GPU or SIMD implementation. PolarQuant and TurboQuant reconstruct approximate vectors for scoring in NumPy, while optimized implementations would use packed codes and fused kernels. Thus, retrieval latency numbers are useful for relative debugging but should not be treated as production throughput.
- **Single model and dataset.** We evaluate CLIP ViT-B/32 on Flickr30k. Larger CLIP backbones, multilingual CLIP variants, or out-of-domain image-text corpora may show different sensitivity.
- **Limited query sample.** We use 1,000 queries per task per seed, which is sufficient for Recall@10 trends but less detailed for rare failure cases and Recall@1 tail behavior.
- **No OPQ or optimized ANN search.** FAISS-PQ is a strong baseline, but optimized product quantization, OPQ, and approximate search indexes could further shift the Pareto frontier.

7 Conclusion and Future Work

We evaluated QJL, PolarQuant, TurboQuant, and FAISS-PQ on CLIP multimodal retrieval. The central conclusion is that cross-modal retrieval is the most compression-sensitive axis. Same-modal tasks can remain strong even at 1 bit per dimension, while text-to-image and image-to-text retrieval degrade sharply. This supports our first hypothesis and suggests that future evaluations of embedding compression should report cross-modal recall explicitly rather than relying only on same-modal nearest-neighbor preservation.

Among compressors, FAISS-PQ and PolarQuant are strongest in the 2–4 bit regime. TurboQuant is theoretically well motivated and passes synthetic distortion checks, but it does not dominate on CLIP retrieval. This does not mean TurboQuant is ineffective; rather, it suggests that CLIP’s modality gap and ranking-sensitive objective are not fully captured by generic distortion guarantees.

Future work should test larger CLIP models, evaluate out-of-domain retrieval, add optimized packed kernels for PolarQuant and TurboQuant, and explore modality-aware quantization. One promising direction is to allocate different bit budgets to image and text embeddings, or to preserve the modality-gap direction explicitly during compression.

Project Artifacts

The code repository and experiment artifacts are available at:

- Project repository: <https://github.com/QUASARS06/turboquant-clip>
- Google Colab embedding notebook: [PASTE_COLAB_LINK_HERE](#)
- Results/figures folder or release: [PASTE_RESULTS_LINK_HERE](#)

Reproducibility

The full experiment can be reproduced with the following pipeline:

```
python -m src.embed encode --out-dir data
```

Encodes Flickr30k images and captions with CLIP ViT-B/32 and saves raw embedding outputs.

```
python -m src.embed finalize --out-dir data
```

Finalizes and L2-normalizes the saved image and text embeddings.

```
python -m src.eval.experiment --data-dir data --out-dir results \
    --seeds 0,1,2,3,4 --max-queries 1000
```

Runs the full compressor sweep over methods, bit-widths, retrieval tasks, and random seeds.

```
python -m src.analysis.plots --results-dir results
```

Generates the cross-modal versus same-modal plot and the memory-accuracy tradeoff plot.

```
python -m src.analysis.geometry --data-dir data --out-dir results
```

Computes and plots CLIP coordinate statistics before and after random rotation.

```
python -m src.analysis.failure_modes --data-dir data --results-dir results
```

Extracts worst-query examples for qualitative failure-mode analysis.

The compressor validation tests are:

```
python -m pytest -s tests/test_compressors.py
```

A Appendix: Use of LLM Assistance

This appendix is reserved for prompts and responses used to assist with implementation review, debugging, and report drafting. All final code and experimental conclusions were verified by local tests and rerun experiments.

References

- [1] Alec Radford et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.
- [2] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2011.
- [3] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.
- [4] Amir Zandieh, Majid Daliri, and Insu Han. QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead. *AAAI*, 2025.

- [5] Insu Han, Praneeth Kacham, Amin Karbasi, Vahab Mirrokni, and Amir Zandieh. PolarQuant: Quantizing KV caches with polar transformation. AISTATS, 2026.
- [6] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. ICLR, 2026.
- [7] Weixin Liang et al. Mind the gap: Understanding the modality gap in multi-modal contrastive representation learning. NeurIPS, 2022.
- [8] tonbistudio. turboquant-pytorch: From-scratch PyTorch implementation of TurboQuant. <https://github.com/tonbistudio/turboquant-pytorch>, 2026.
- [9] Memoriant. DGX Spark KV cache benchmark. <https://github.com/Memoriant/dgx-spark-kv-cache-benchmark>, 2026.
- [10] ggml-org. TurboQuant — Extreme KV cache quantization discussion. <https://github.com/ggml-org/llama.cpp/discussions/20969>, 2026.